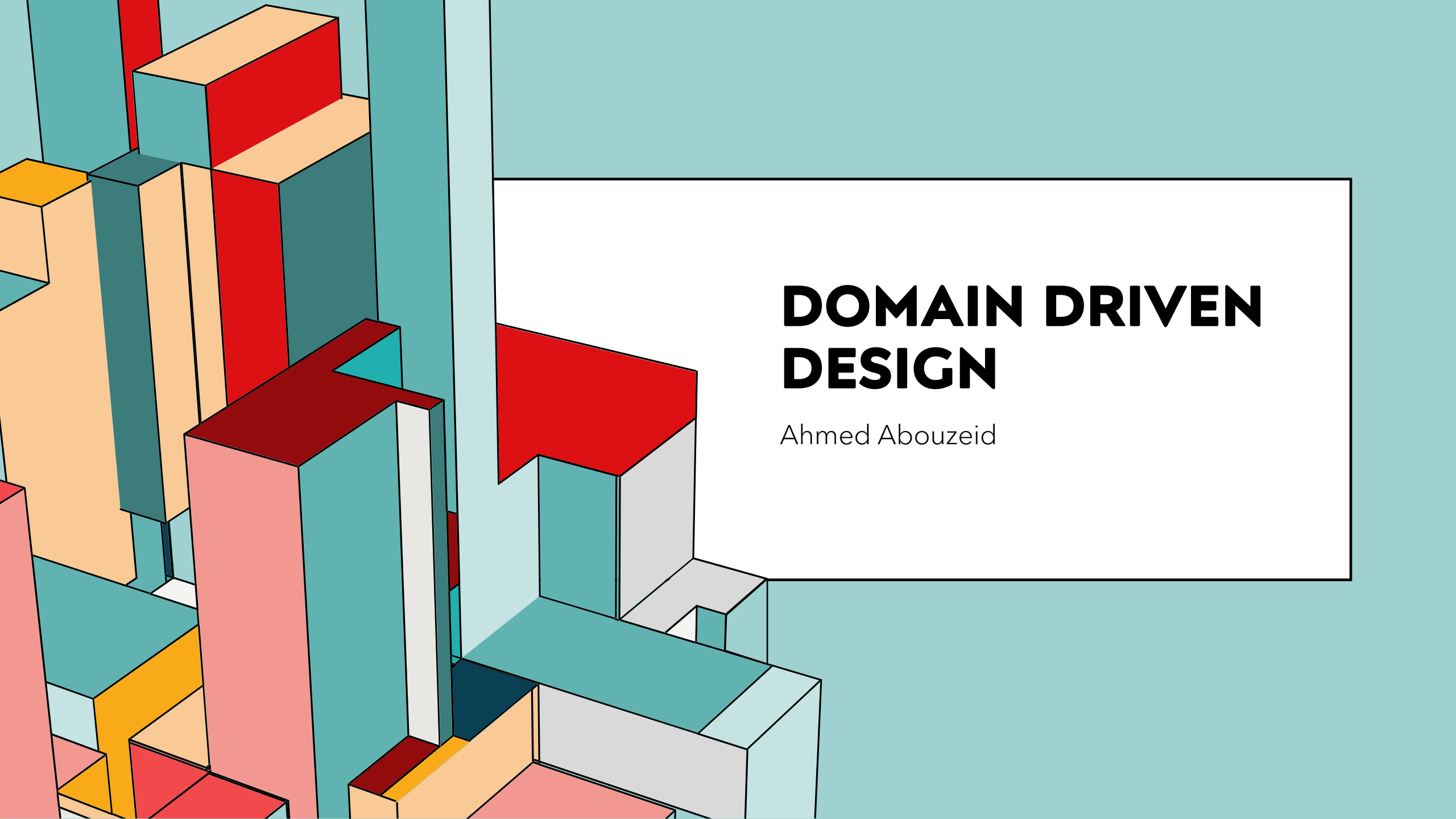


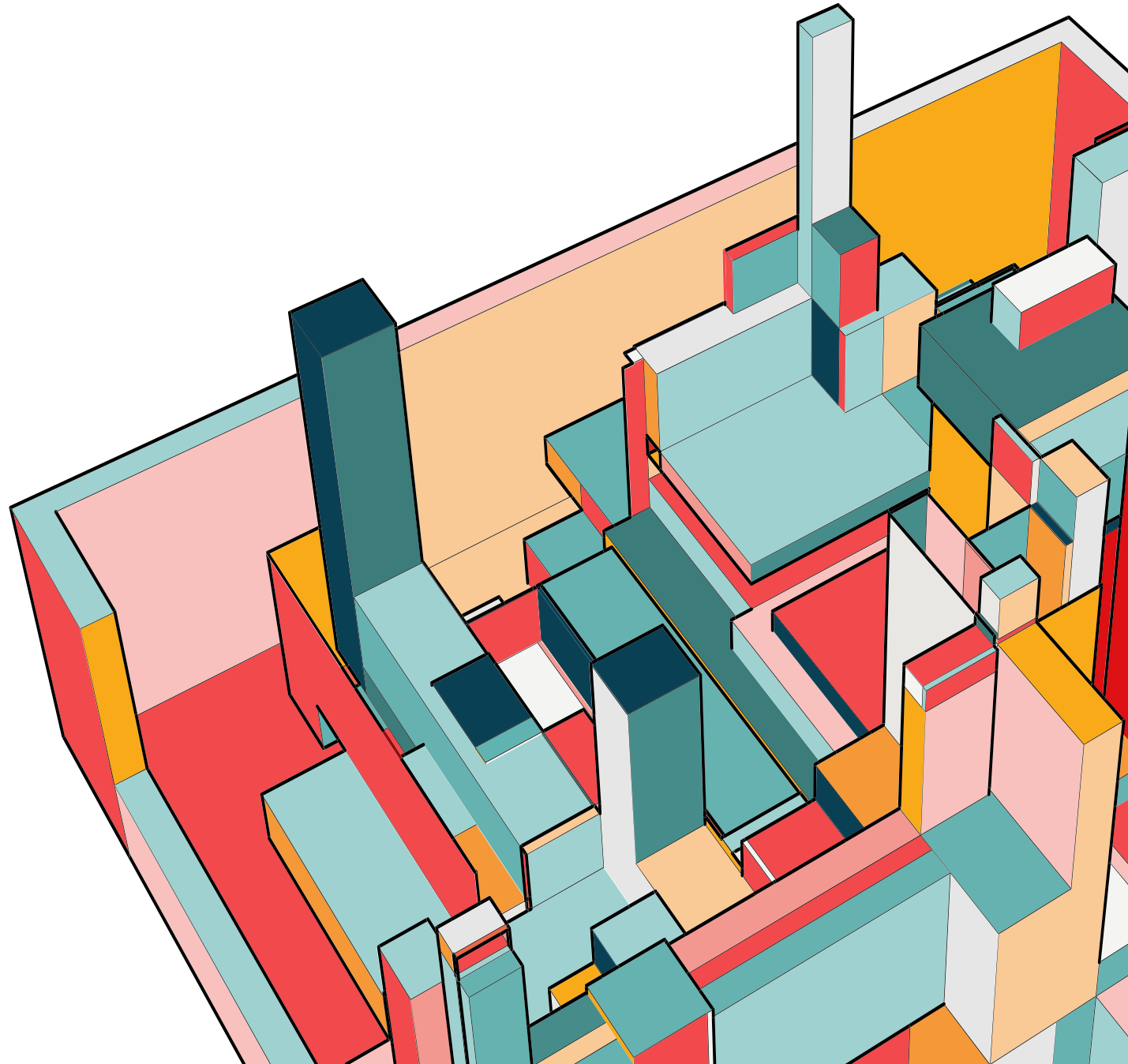


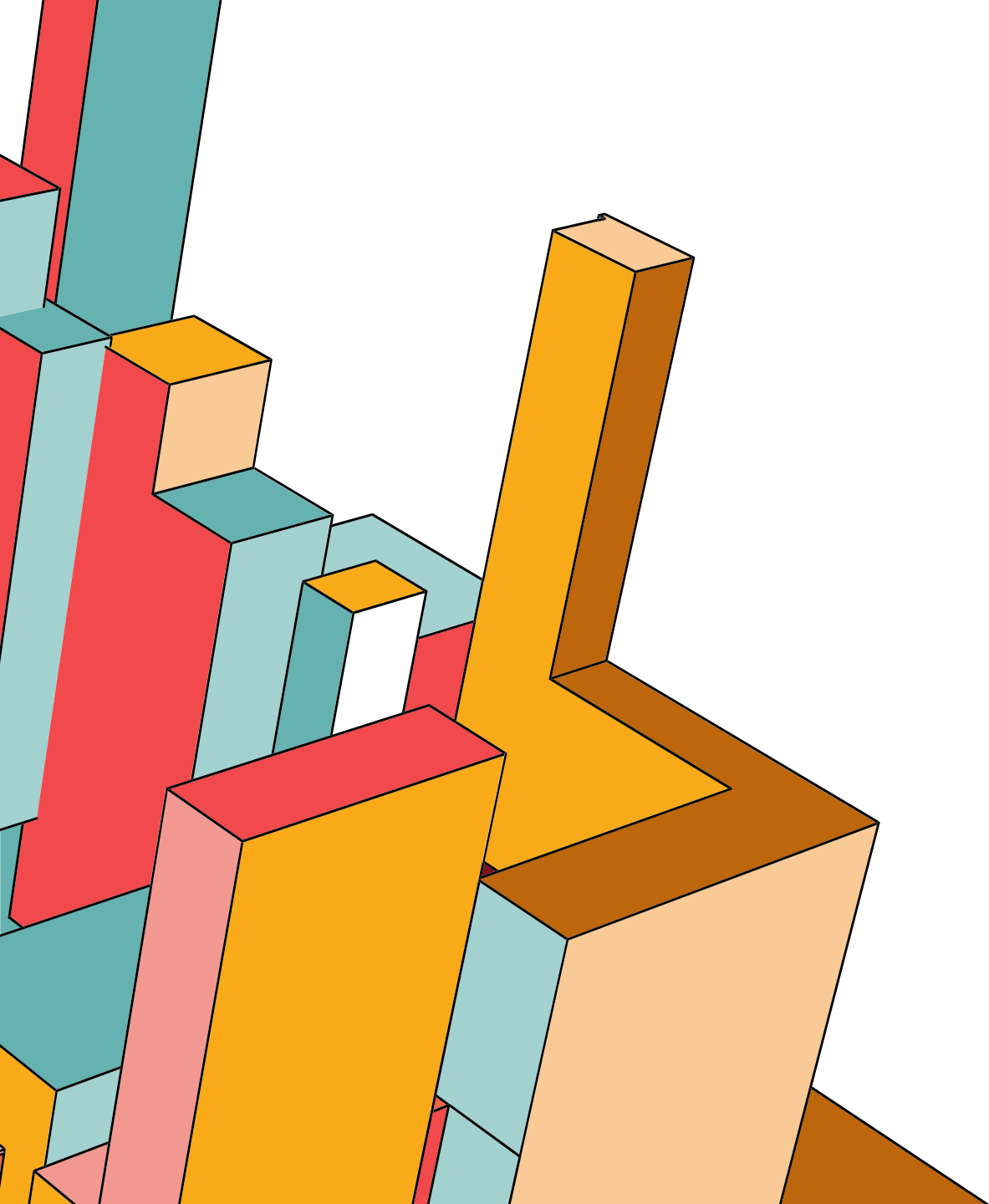
DOMAIN DRIVEN DESIGN

Ahmed Abouzeid

AGENDA

- What is Domain Driven Design?
- Modeling the problem
- Element of DDD

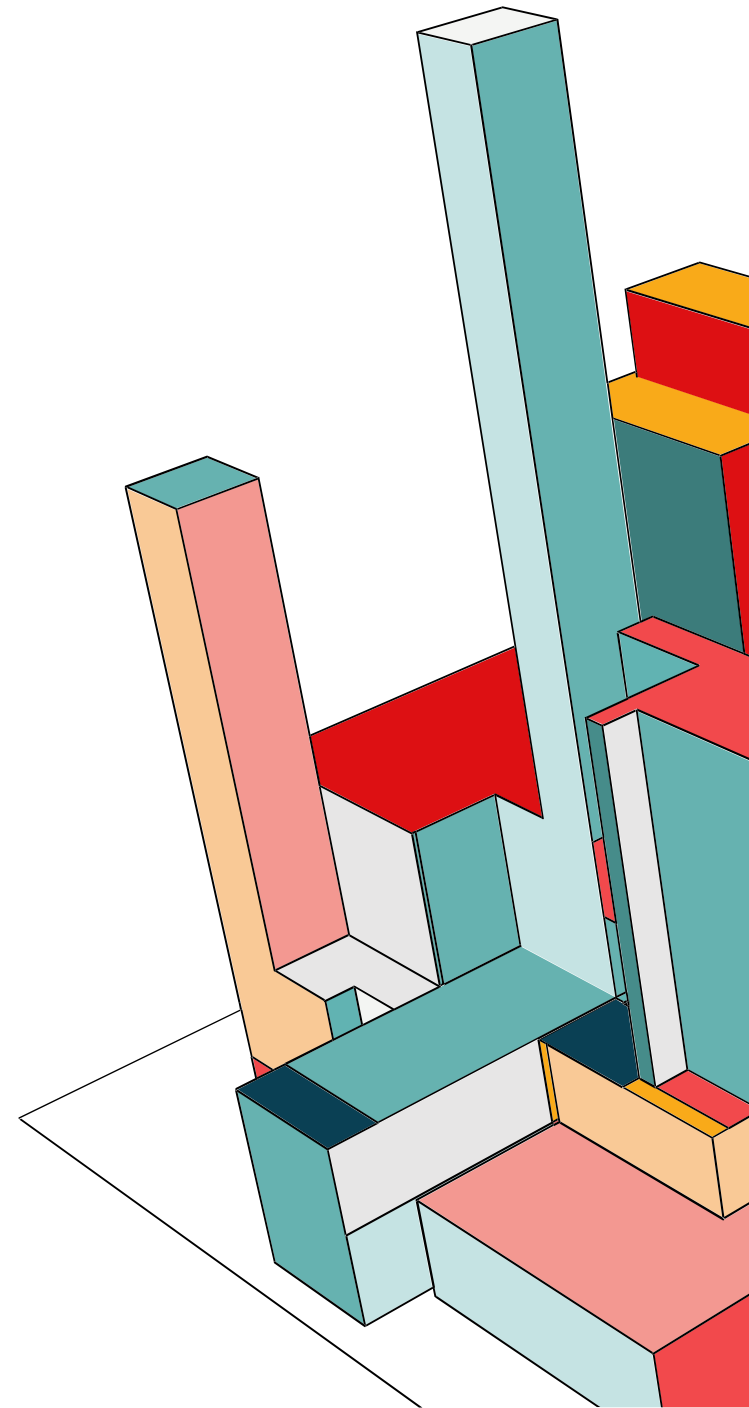




WHAT IS DOMAIN DRIVEN DESIGN?

SOFTWARE COMPLEXITY TYPES

- Technical complexity
- Amount of data
- Performance
- Business logic complexity

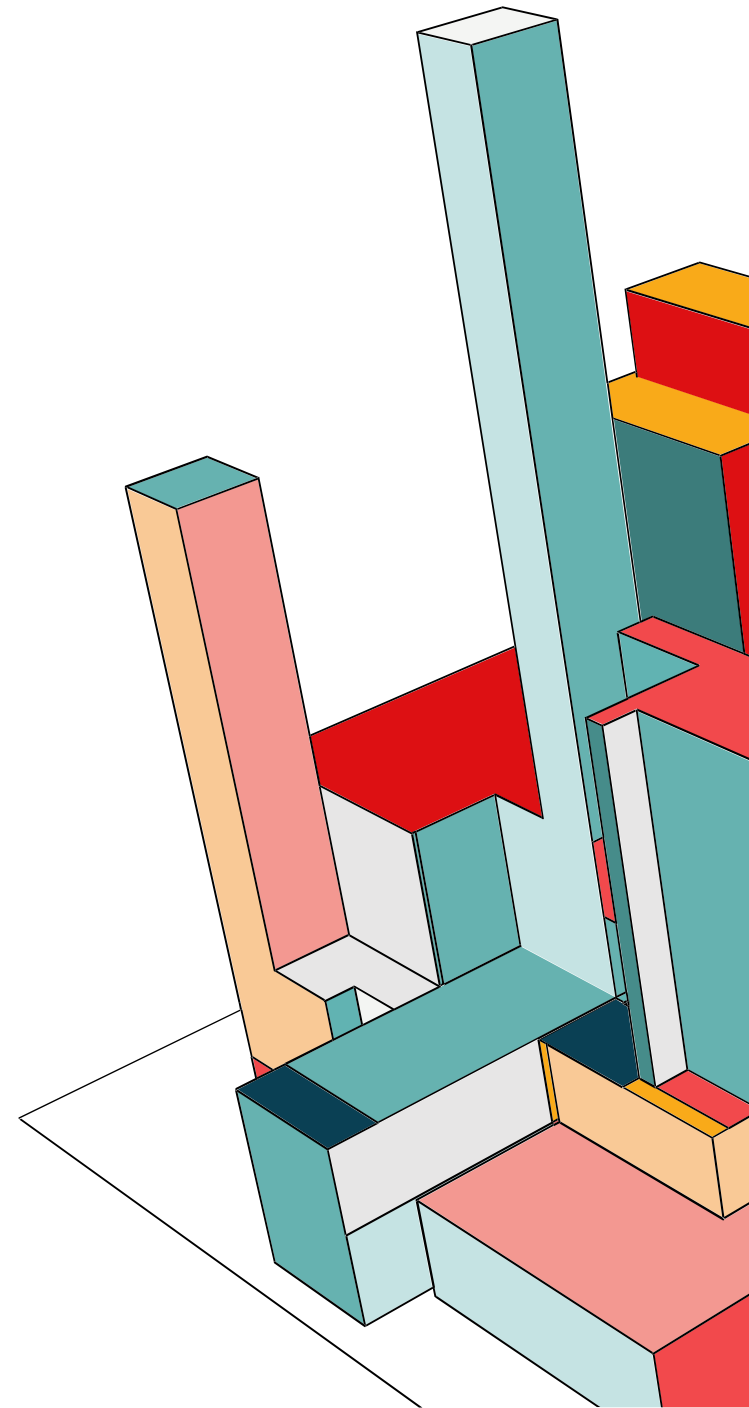


SOFTWARE COMPLEXITY TYPES

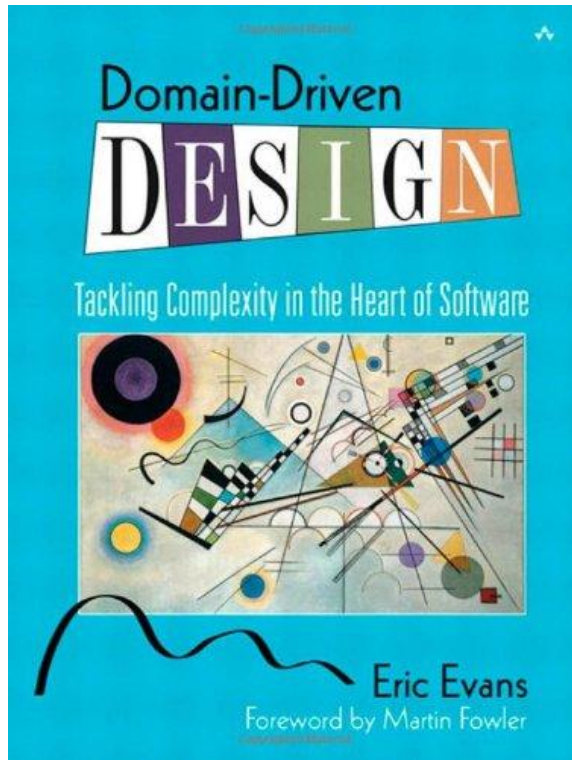
- Technical complexity
- Amount of data
- Performance
- Business logic complexity → DDD

Domain Driven Design can handle complex business logic complexity in such way that it would be possible to extend, maintained and keep it simple

Patterns & Practices

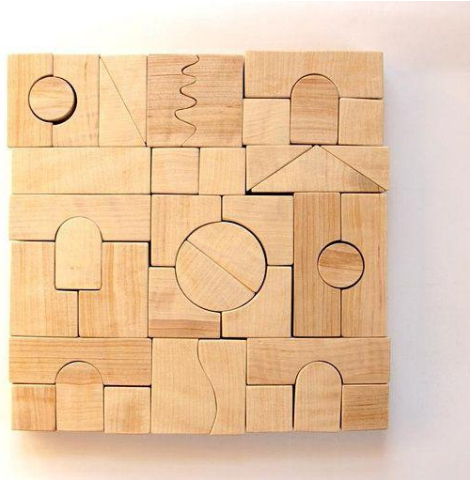


THE BLUE BOOK



Domain Driven Design - Tackling Complexity in the Heart of Software by Eric Evans is the main reference for Domain Driven Design

WHAT IS DOMAIN DRIVEN DESIGN ?



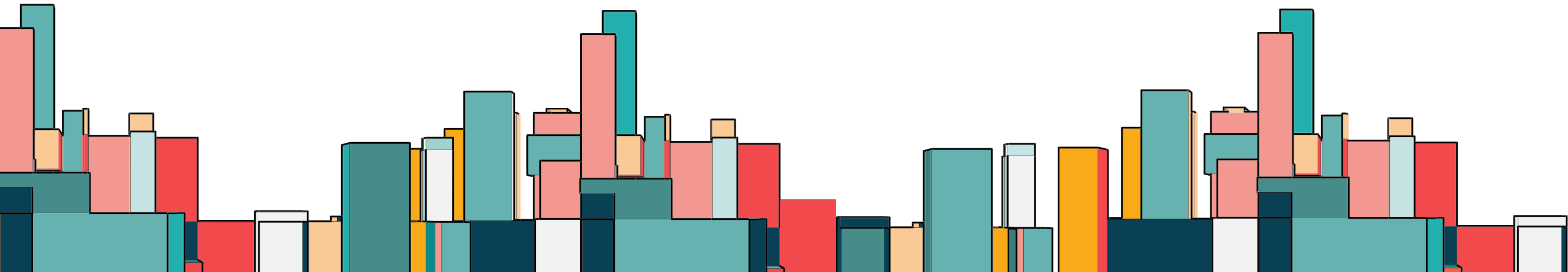
Domain

Field of study that defines a set of common requirements, terminology, and functionality for any software program

Domain Driven Design (DDD)

The expansion upon and application of the domain concept

It aims to ease the creation of complex applications by connecting the related pieces of the software into an ever-evolving model



UBIQUITOUS LANGUAGE



Domain
Expert

Balance
Sheet

Budget

VAT

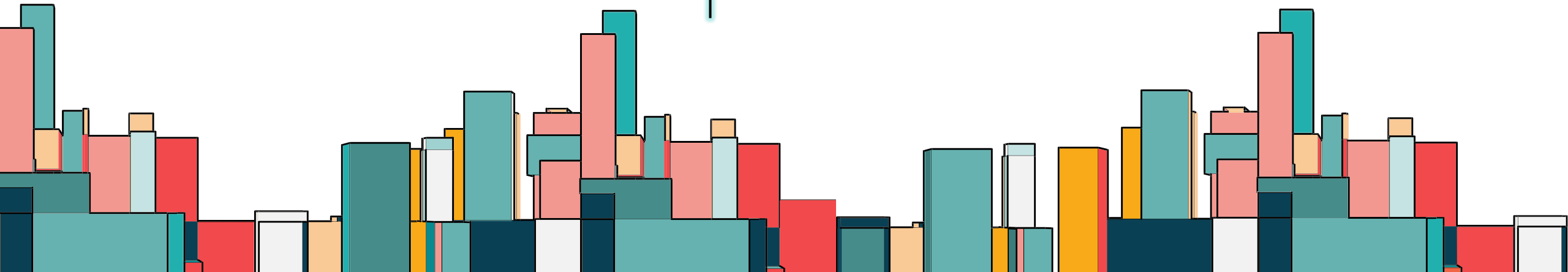
Classes &
Objects

Database
Tables

HTML
Views



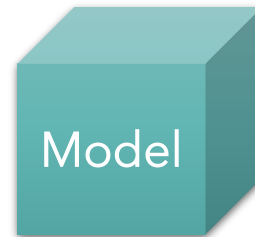
Developer



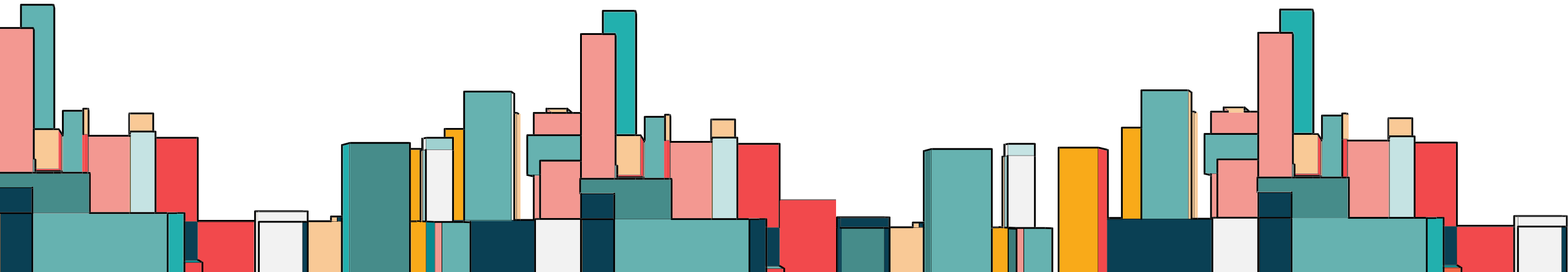
UBIQUITOUS LANGUAGE



Domain
Expert



Developer

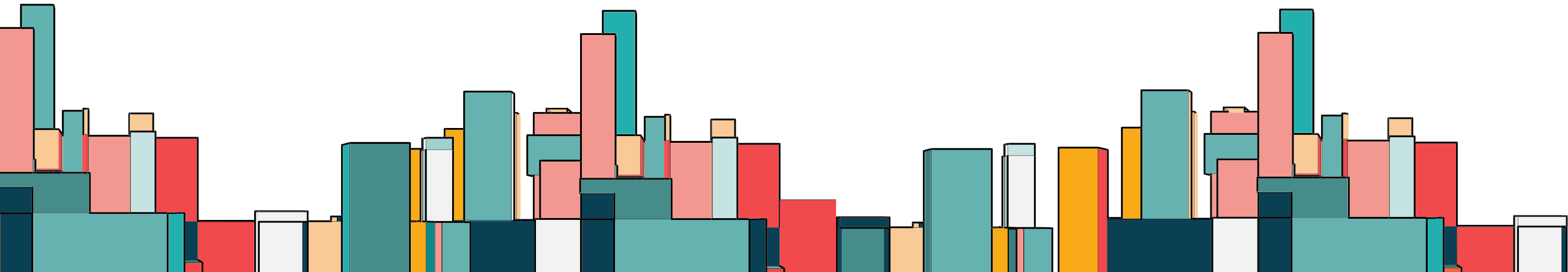


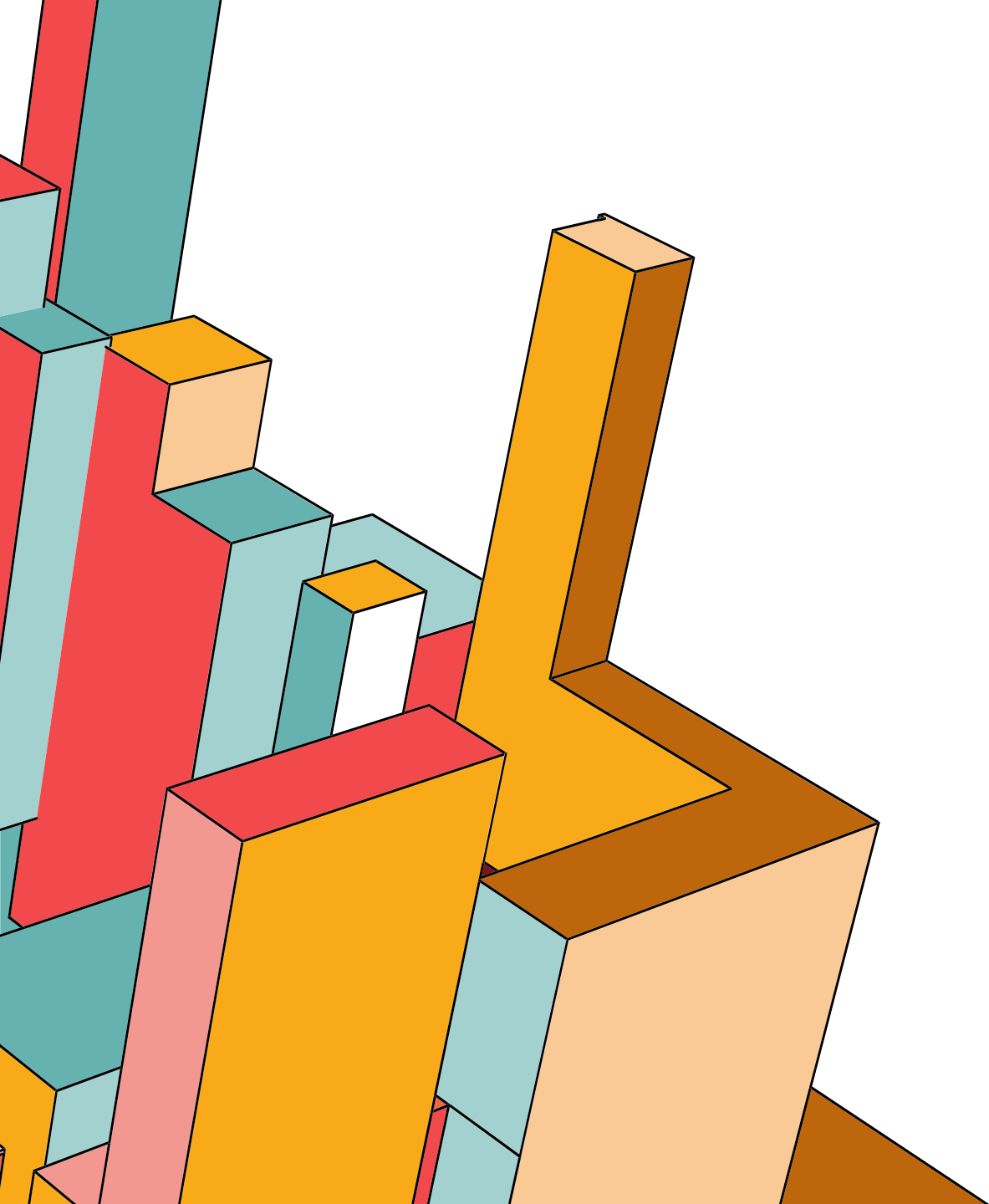
UBIQUITOUS LANGUAGE

CUSTOMER CONVERSATION



- Important to get on the “same page” with the domain expert
- The customer gained better understanding of their business process by describe it in terms we could understand and model
- Avoid speaking in programmer terms
- Focus on how the domain works, not how the software will work
- Make the implicit knowledge of domain expert explicit

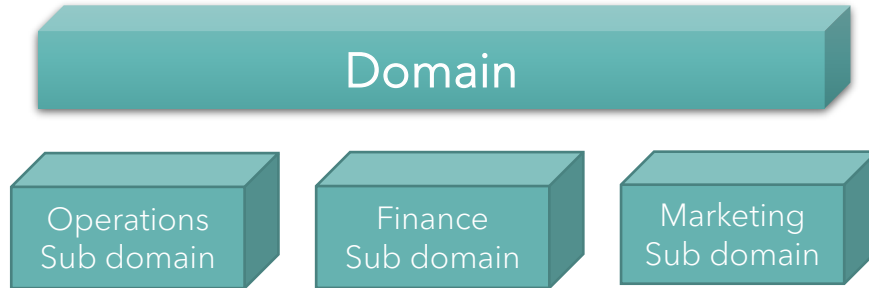




MODELING THE PROBLEM

MODELING THE PROBLEM

Problem Space



We have to find clear boundaries between different parts of the system

Sub Domain is a problem space concept

Bounded Context is a solution space concept

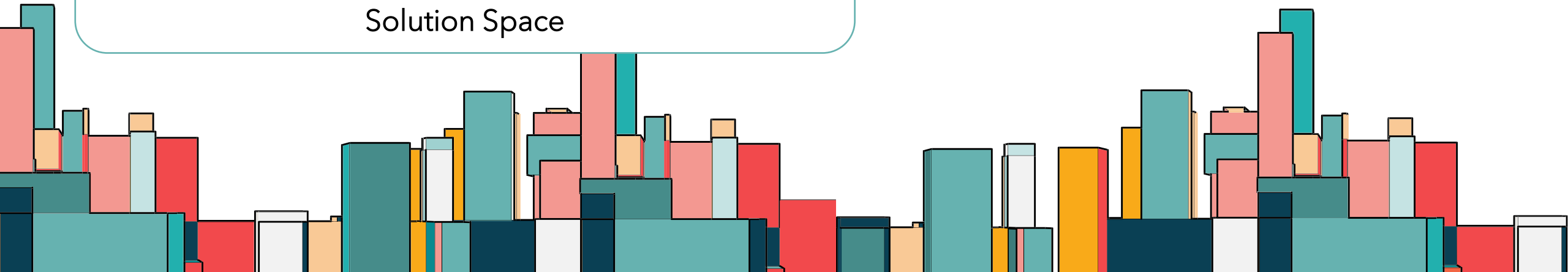
Operations Bounded Context

Full Search Bounded Context

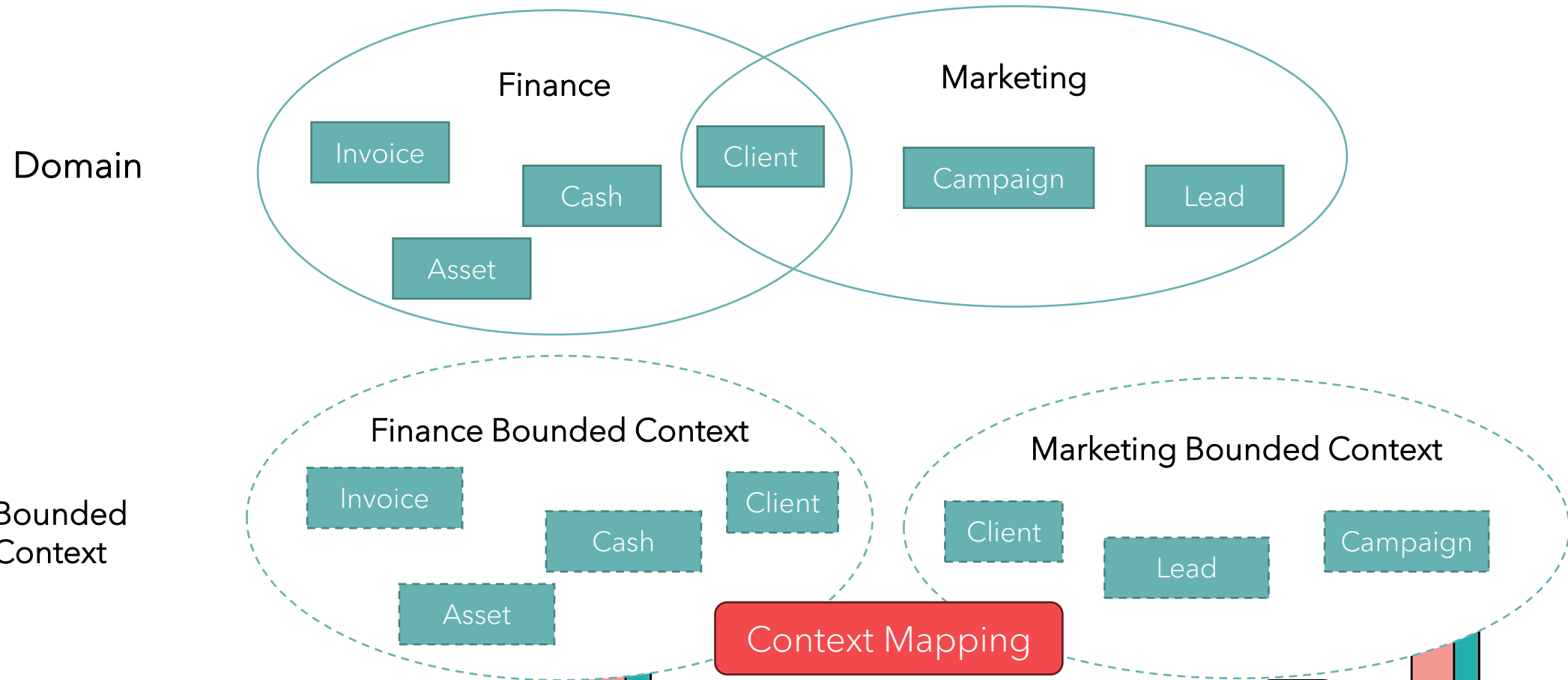
Finance Bounded Context

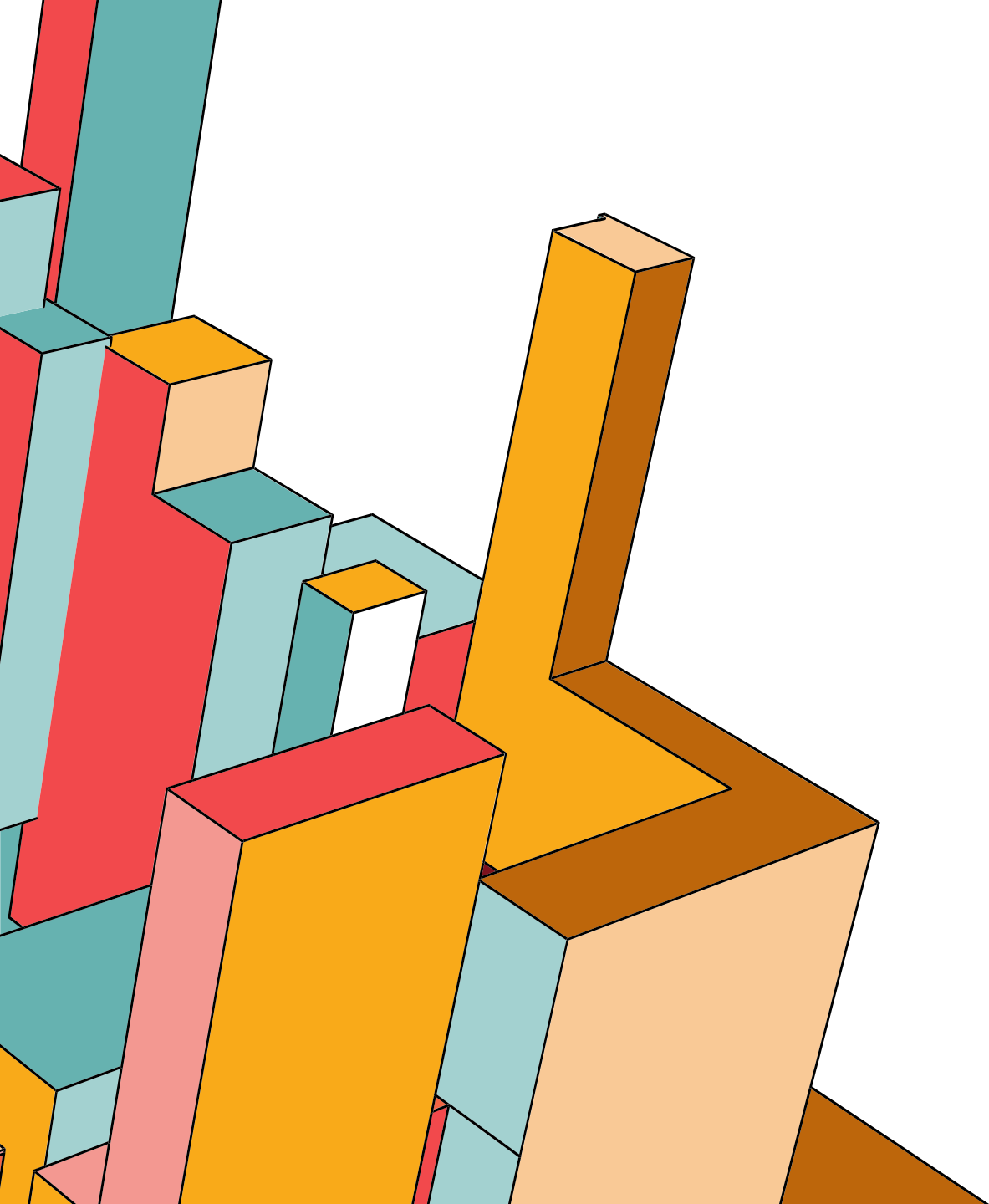
Marketing Bounded Context

Solution Space



MODELING THE PROBLEM





ELEMENTS OF DDD

FOCUS ON DOMAIN

- Focus on **Domain Model** not how to persist
- Focus on **behavior** not properties

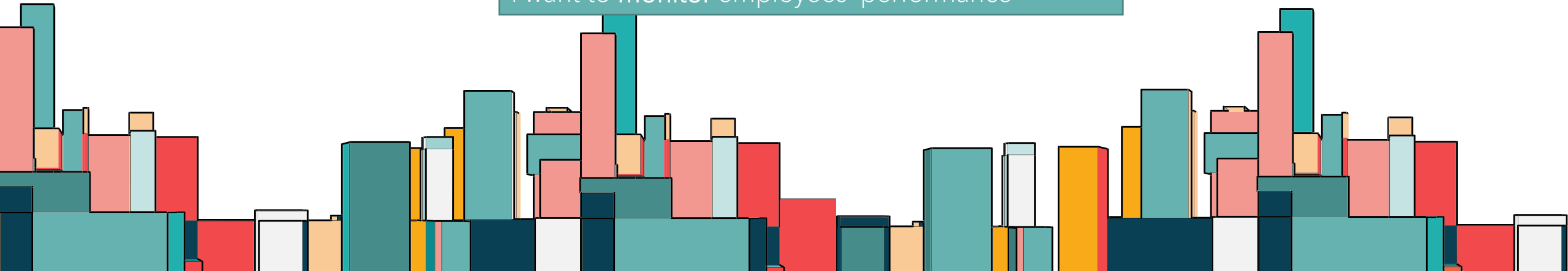


I want to **purchase** a product

I want to **approve** the workflow request

I want to **schedule** a meetings

I want to **monitor** employees' performance



ANEMIC AND RICH MODELS

Anemic Model

- Bag of setters and getter
- No Behaviors included
- Considered as Anti-pattern
- Bulky services

```
public class Client
{
    public long Id { get; set; }

    public string FirstName { get; set; }

    public string LastName { get; set; }

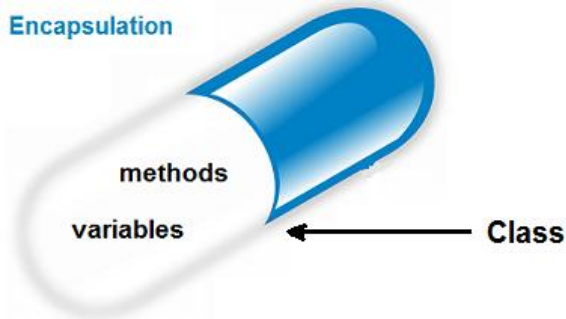
    public string Status { get; set; }

    public ICollection<Invoice> Invoices { get; set; }
}
```

Rich Model

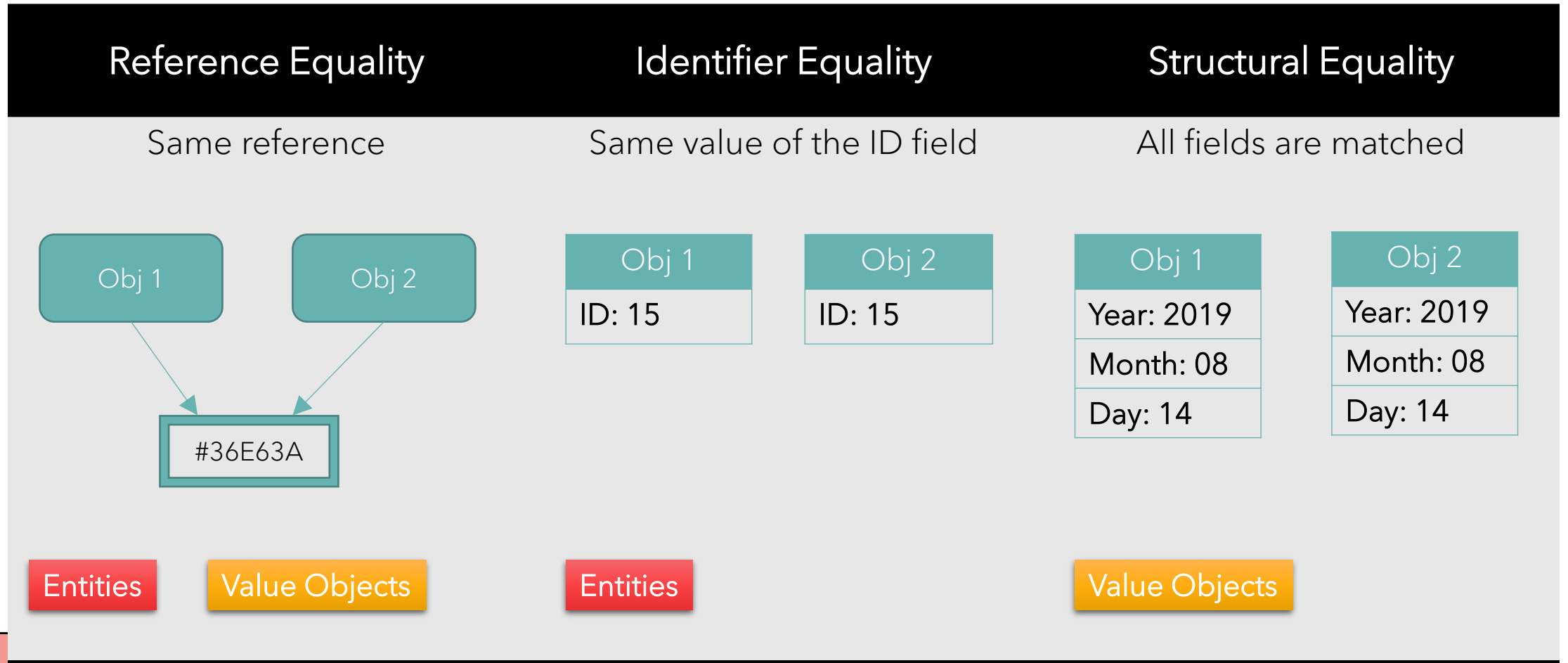
- Business logic inside domain models
- Apply OOP encapsulation principle
- Thin services
- Services only coordinate and delegate tasks to domain models

Encapsulation



ENTITIES VS. VALUE OBJECTS

Types of Equality



ENTITIES VS. VALUE OBJECTS

Entities

- Identity based on ID field
- Mutable
- May have methods
- Continuum
- Represent system state

Value Objects

- Identity based on composition of the values
- MUST be Immutable
- May have methods
- Have a zero lifespan
- Have a meaning only in the context of an entity

ENTITIES VS. VALUE OBJECTS

Entities

- Identity based on ID field
- Mutable
- May have methods
- Continuum
- Represent system state

Single Responsibility is a good principal to apply to entities

Demo: Entity in Code

ENTITIES VS. VALUE OBJECTS

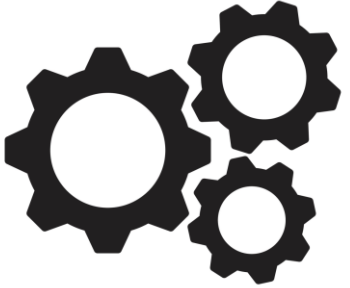
Examples: string & DateTime

Demo:
Address Value Object

Value Objects

- Identity based on composition of the values
- MUST be Immutable
- May have methods
- Have a zero lifespan
- Have a meaning only in the context of an entity

DOMAIN SERVICES

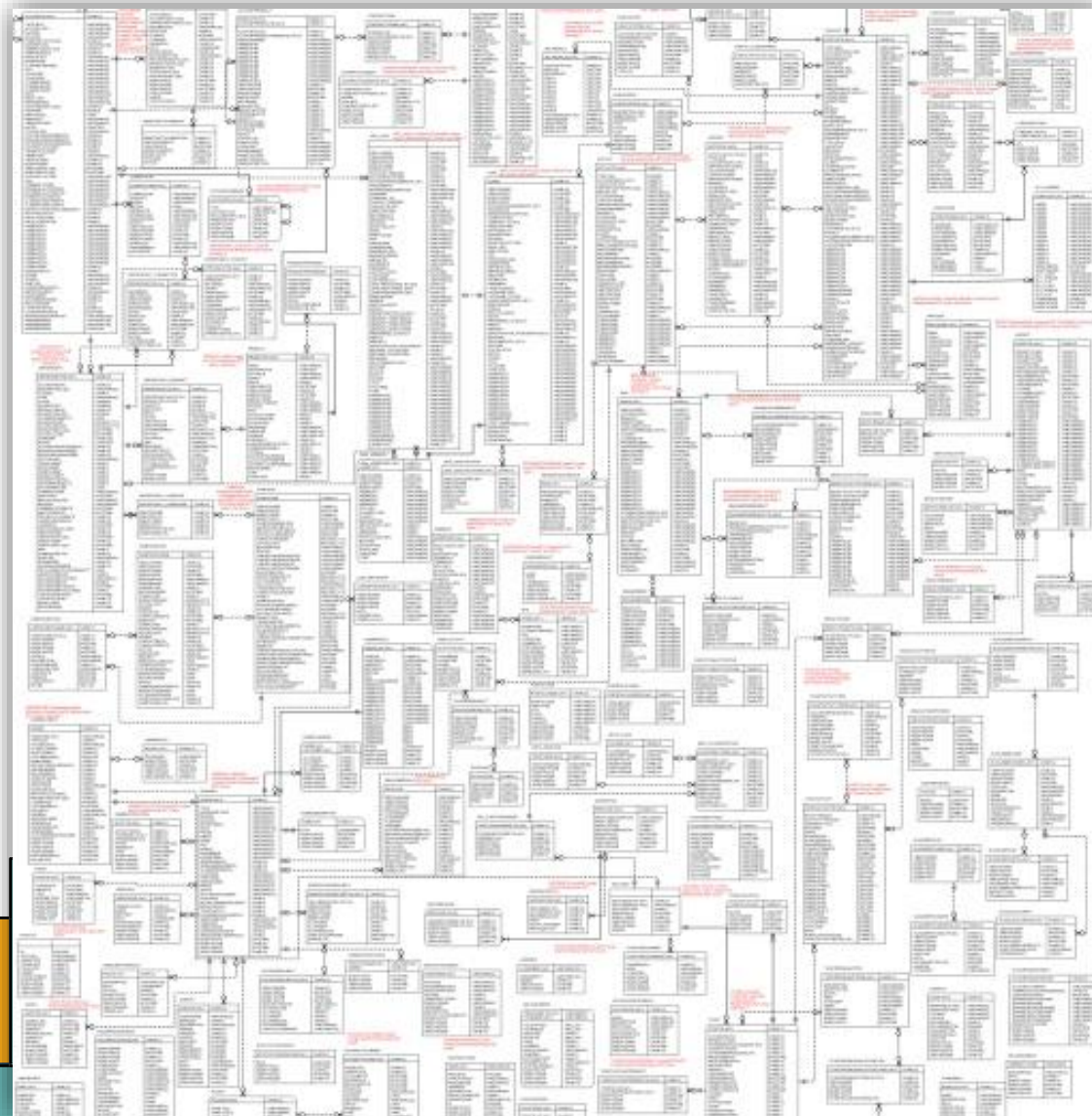


- Stateless classes
- Encapsulates domain logic or operations that don't naturally fit within the boundaries of an Entity or a Value Object.
- Example: calculate the shipping cost for an order

Demo: Shipping Calculator

AGGREGATES

Big Ball of Mud



AGGREGATES

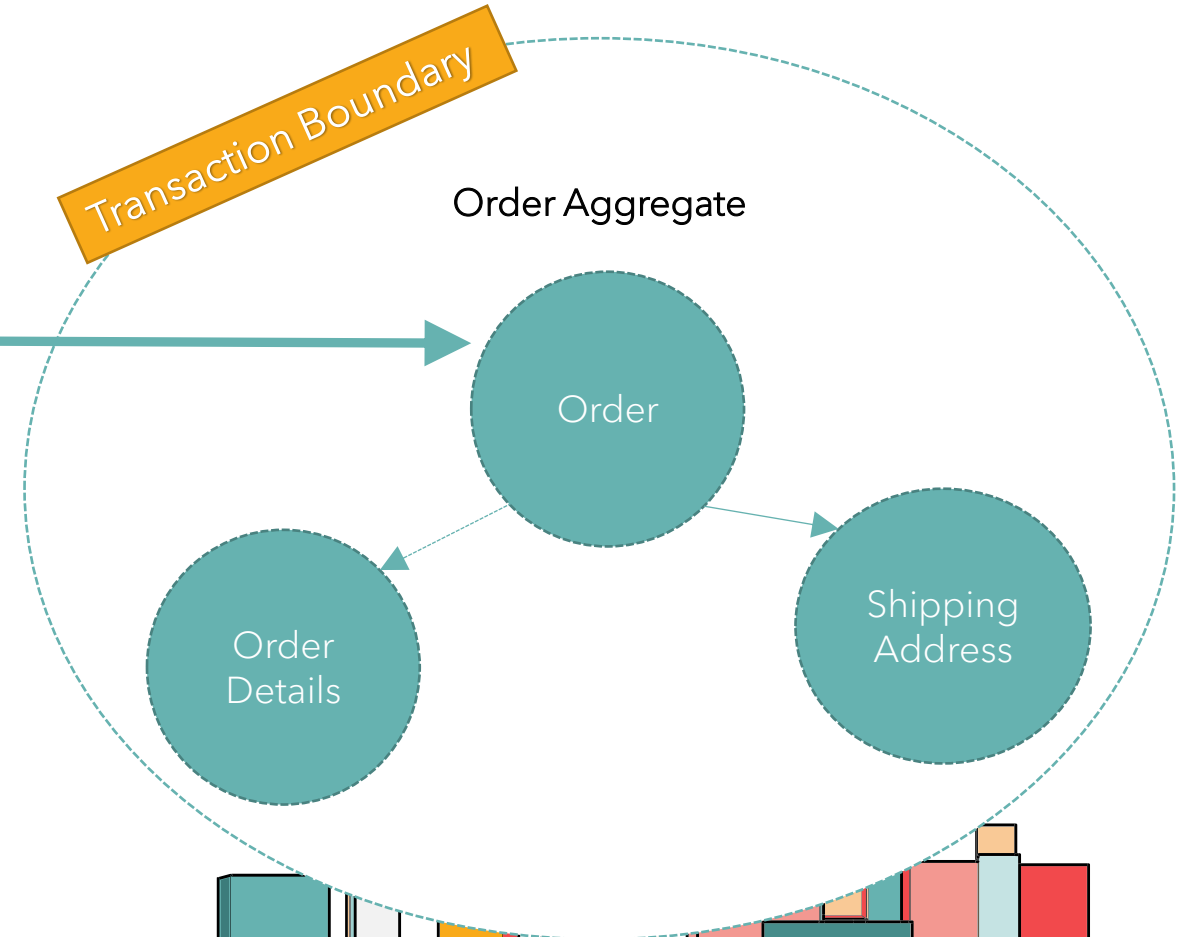
Aggregate is a cluster of associated objects that we treat as a unit for the purpose of data changes

Aggregate Root

Aggregate Root ensure the integrity of the aggregate as a whole

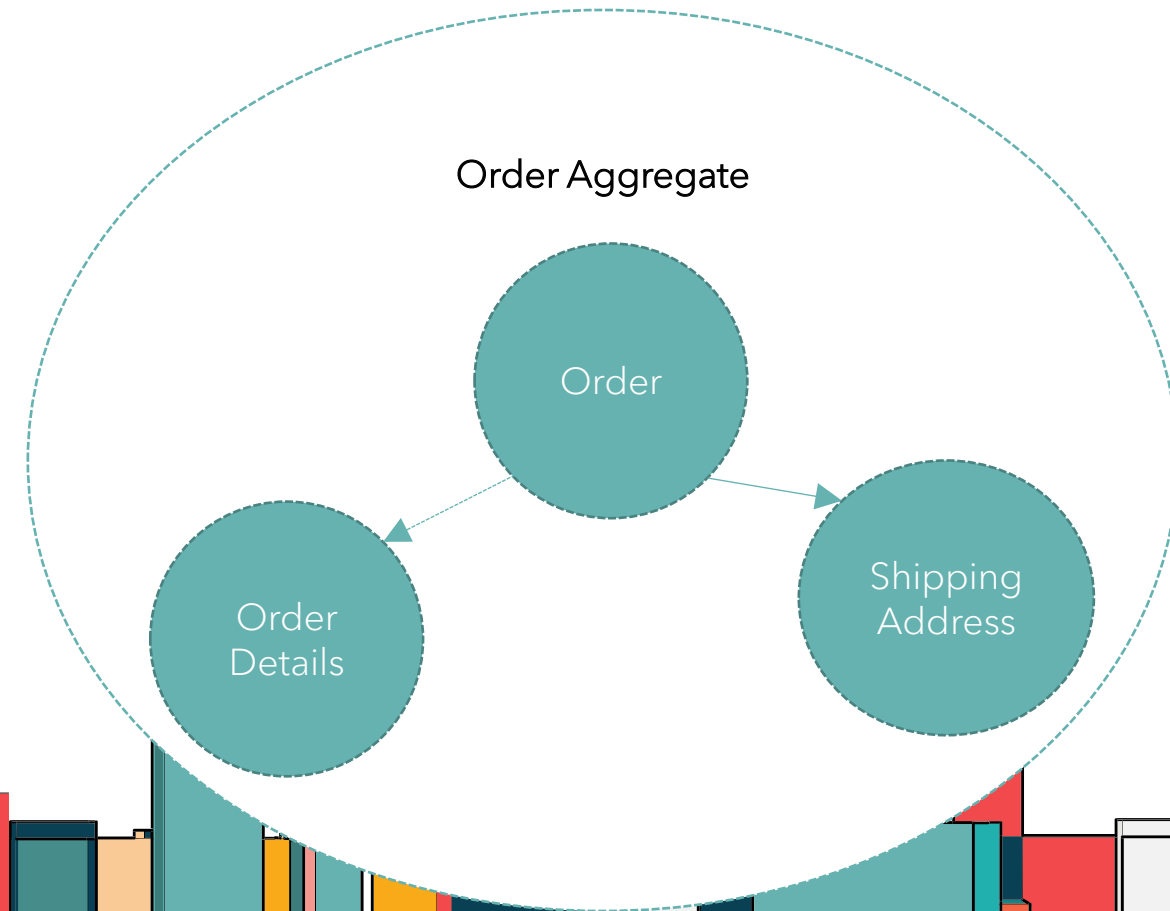
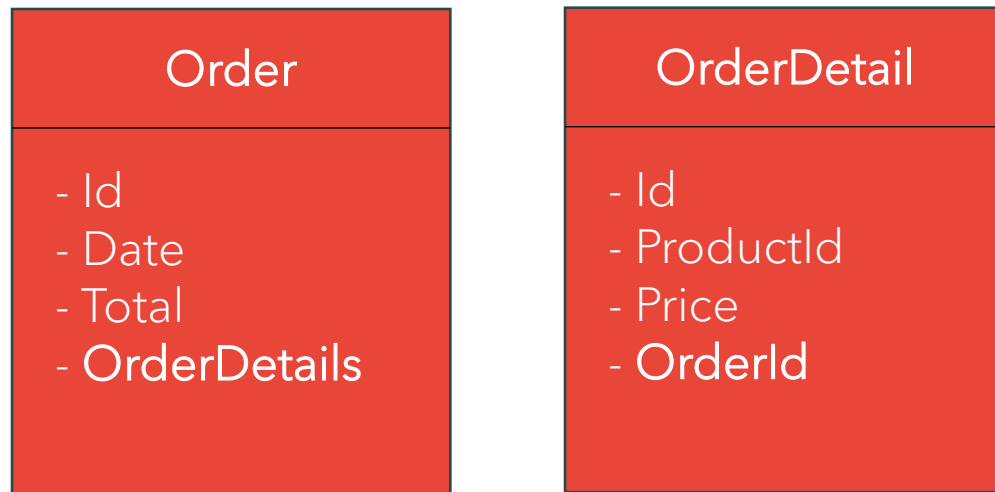
If you delete the Aggregate Root, delete the others

Aggregates of one are acceptable



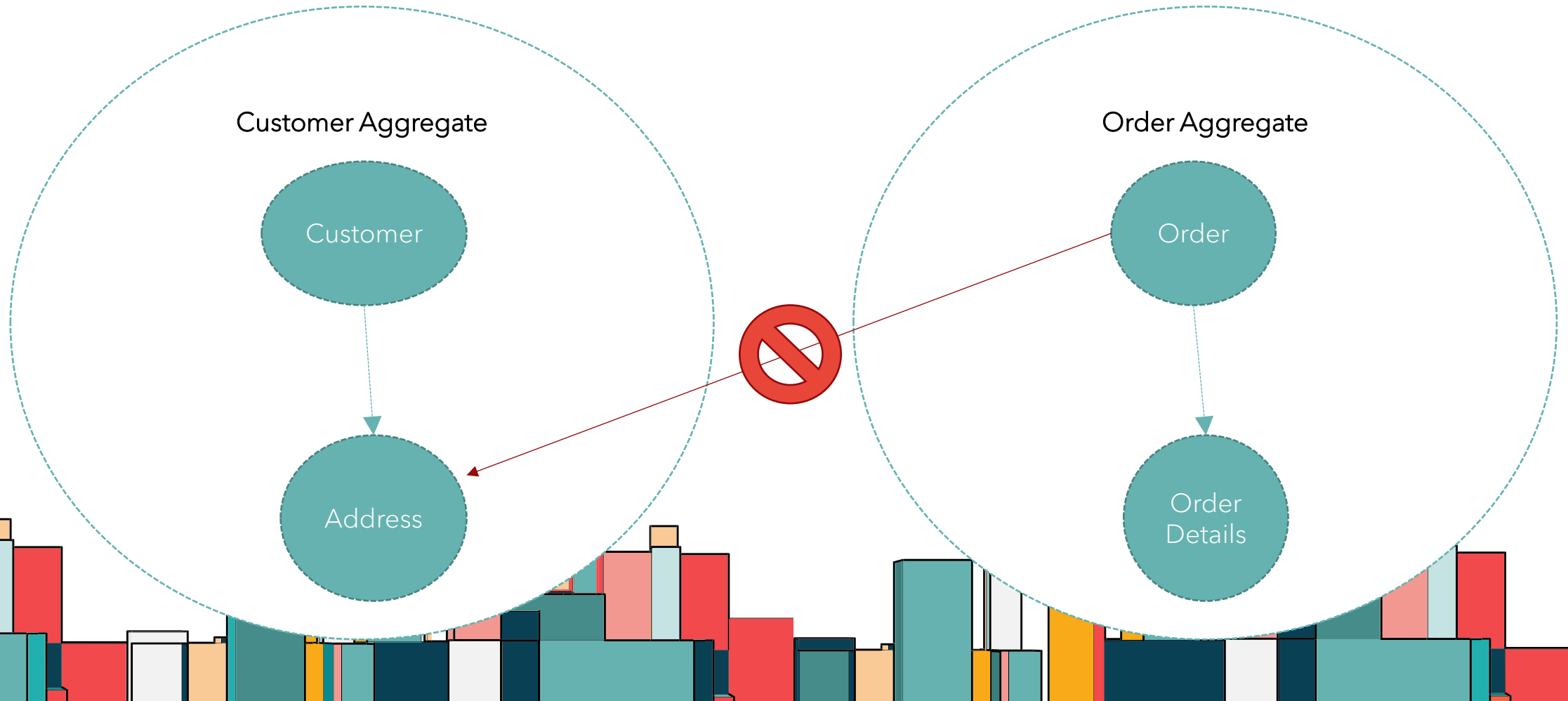
AGGREGATES

- A bidirectional association means that both objects can be understood only together.
- One-way traversal reduces interdependence and simplifies the design



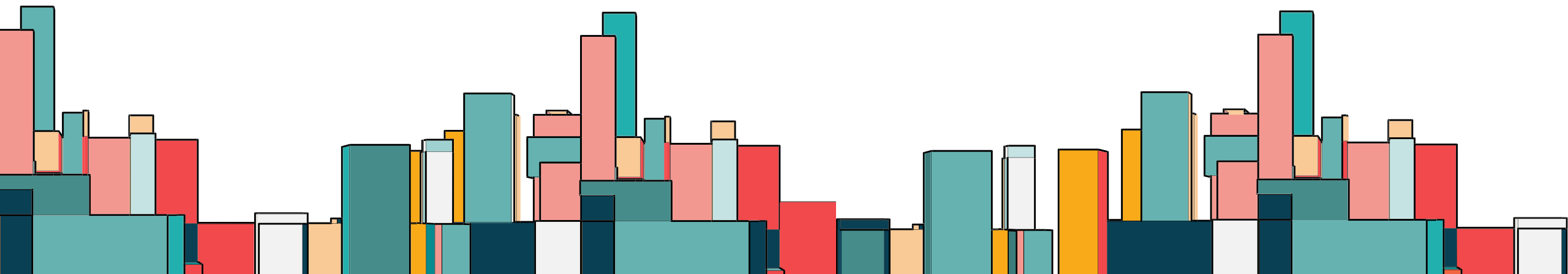
AGGREGATES

- Enforce the boundaries by prohibiting direct reference to object within an aggregate that are not the root of the aggregate



AGGREGATES & INVARIANTS

- A condition that should be always be true for the system to be in a consistent state
 - Total items on order do not exceed limit
 - Two appointments do not overlap
 - End date should follow the start date
- Aggregate root is the only responsible to keep the aggregate in a consistent state



REPOSITORIES

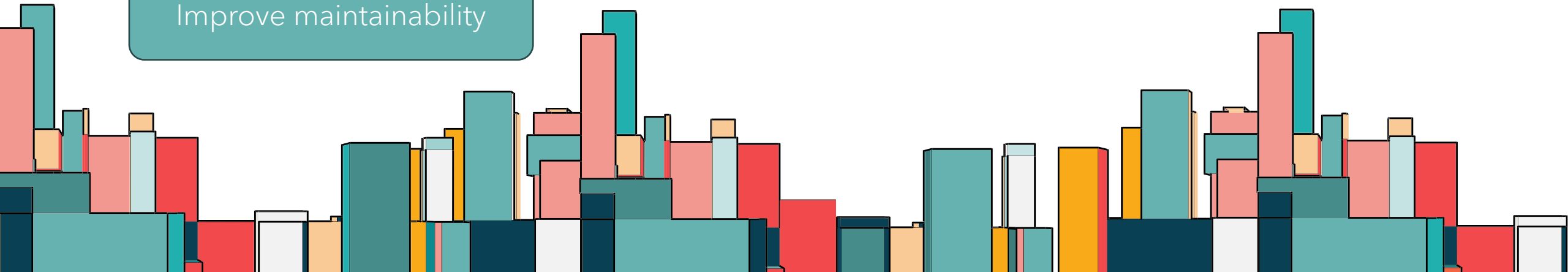
- Provides a structured and standardized way to access, manage, and manipulate data while abstracting the underlying details of data storage technologies.
- Persistence Ignorance: Business objects have no logic related to how data is stored and retrieve

Common abstraction for
persistence

Separation of concerns

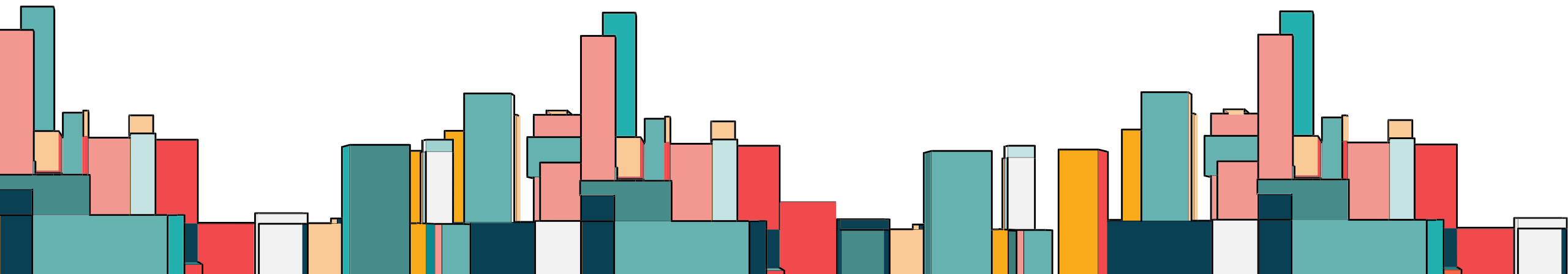
Enable testability

Improve maintainability



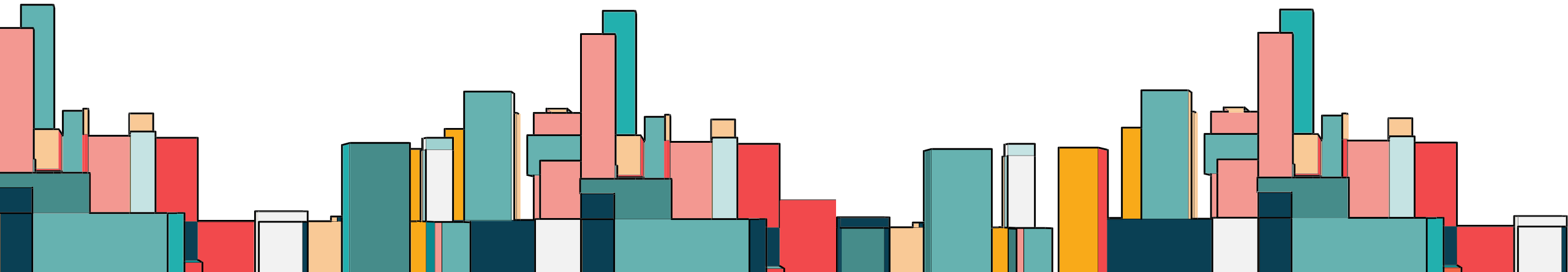
REPOSITORIES

- Think of it as an in-memory collection
- Only for aggregate roots
- IQueryable !
- CQRS – Command and Query Responsibility Segregation



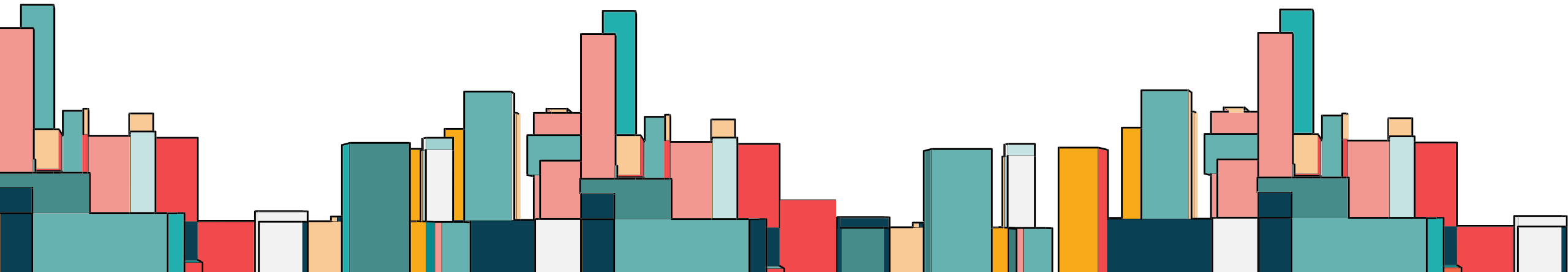
APPLICATION SERVICES

Orchestrate the execution of domain logic and don't themselves implement any domain logic



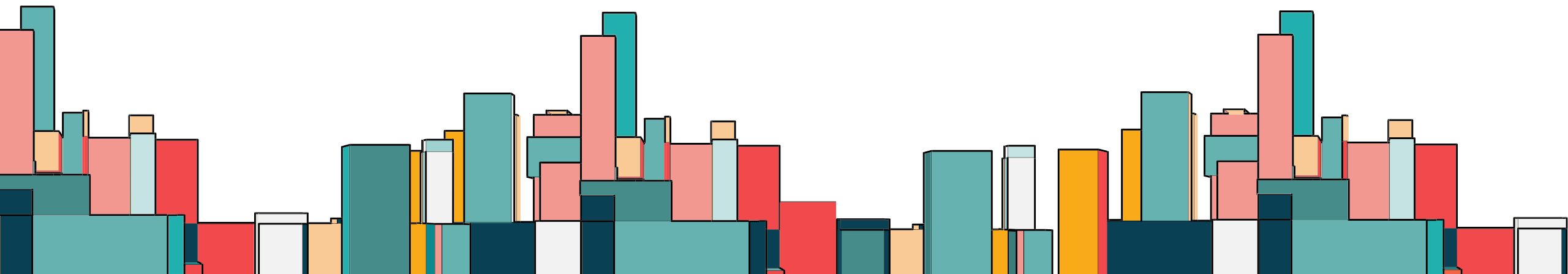
DOMAIN EVENTS

- User a domain event to capture an occurrence of something that happened in the domain
- Each Domain Event is its own class



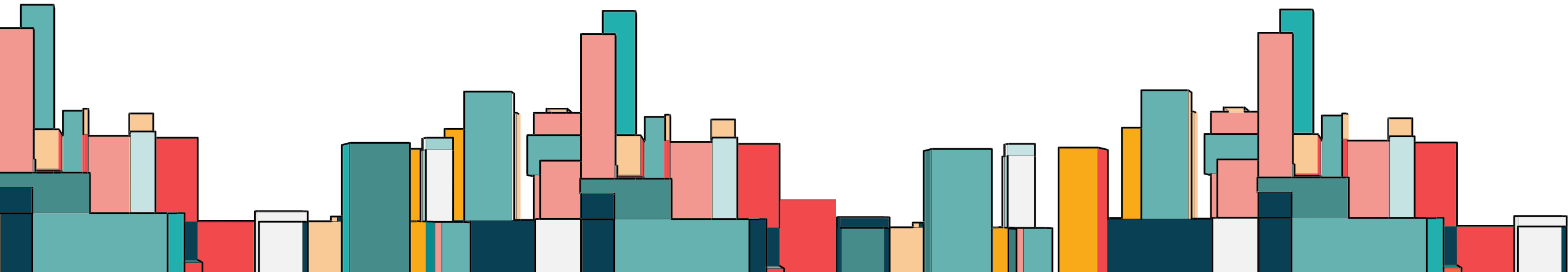
ANTI-CORRUPTION LAYER

- acts as a bridge between two different domains and ensures that each bounded context can maintain its own language and terminology without being affected by the other



RESOURCES

- [Domain-Driven Design Fundamentals | Pluralsight](#)
- [Domain-Driven Design in Practice | Pluralsight](#)
- [Domain-driven Design: Tackling Complexity in the Heart of Software - Eric Evans](#)
- [Domain-Driven Design Distilled](#)
- [Domain Driven Design Topics | martin fowler](#)



THANK YOU

ahmed.abouzeid@outlook.com

